Report

Title: Demonstrating Minimum Spanning Tree (Krushkal's Algorithm)

Report: SY-D-32

Members: Shubhan Punde - 3
Saif Khan - 16
Rahul Sakpal 18
Soham Kasurde - 43

Project Objectives:

Implement Kruskal's Algorithm: Develop a working implementation of Kruskal's algorithm in C/C++ to find the minimum spanning tree (MST) of a given graph.

Graph Representation: Create a data structure to represent the input graph, including nodes/vertices and weighted edges.

Graph Visualization: Use OpenGL to visualize the input graph, highlighting the selected edges of the MST during the algorithm's execution.

User Interaction: Allow user interaction to control the execution of the algorithm, such as starting, pausing, and stepping through the algorithm's steps.

Visual Feedback: Provide visual feedback to the user, such as displaying the progress of the algorithm, the edges under consideration, and the final MST.

Programming Language:

We chose the C programming language due to its efficiency and compatibility with low-level operations necessary for this project. C also offers direct memory management, which is essential for real-time updates.

Graphics Library:

We used the Simple DirectMedia Layer (SDL) graphics library to create the graphical interface. SDL is widely used for 2D graphics in C and offers a platform-independent solution.

Development Environment:

Code::Blocks IDE for C programming.
GCC compiler for code compilation.
GDB debugger for debugging and error resolution.

Project Implementation:

To accomplish the objectives, you can follow these steps:

Data Structures:

Define data structures to represent the graph, edges, and MST.

Kruskal's Algorithm Implementation:

Implement Kruskal's algorithm to calculate the minimum spanning tree. Ensure you have a working algorithm before moving on to visualization.

Graph Visualization:

Create a rendering loop using OpenGL.
Render the graph's nodes and edges, making sure to represent edges with different weights distinctly.
Initially, render the graph without any edges marked as part of the MST.

User Interaction:

Implement user interaction using keyboard or mouse input to control the algorithm's execution. For example, you can use key presses to start and step through the algorithm.

Algorithm Visualization:

Add functionality to highlight edges currently under consideration by the algorithm.
Visualize the progress of the algorithm, showing which edges are being added to the MST.

Display MST:
Once the algorithm completes, render the minimum spanning tree with a different color to distinguish it from other edges in the graph.

Testing and Debugging:
Test your application with various graphs to ensure the correctness of the algorithm and the visualization.

Documentation:
Document your code thoroughly, including comments and explanations of the data structures and algorithms used.

Conclusion:

We successfully developed the MST (minimum spanning tree) solver using Kruskal's algorithm, which will help users calculate the shortest distance between any two points effectively and efficiently.